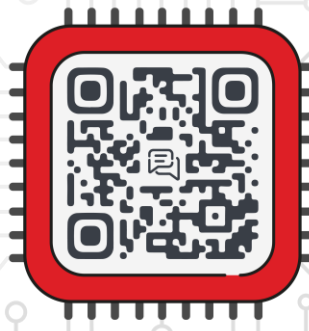




نظري

9

26

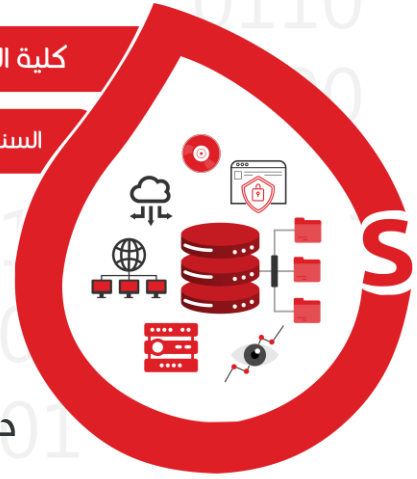
3900 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Concurrency Control

د. مادلين عبود



RB Informatics; 11/12/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

مواضيع المحاضرة:

- البروتوكولات المرتكزة على الأقفال.
- البروتوكولات المرتكزة على الإقفال على مرحلتين.
- البروتوكولات المرتكزة على البيان.
- البروتوكولات المرتكزة على الأختام.
- التعامل مع العرقلة المتبادلة.
- عمليات الإضافة والحذف.
- التزامن في بنية الفهارس.

البروتوكولات المرتكزة على الأقفال Lock-Based Protocols

الأقفال (locks): هي تقنية تستخدم للتحكم بالوصول المتزامن للمعطيات.

يوجد نوعان:

1. القفل الحصري (Exclusive - X Lock):

- يمنع أي عملية أخرى من الوصول إلى البيانات المقفولة (إقفال كلي/تام).
- يستخدم عادةً عند الحاجة إلى تعديل أو تحديث البيانات (عملية write).
- العملية التي تأخذ هذا القفل تكون **الوحيدة** التي تستطيع القراءة والكتابة على البيانات في تلك اللحظة.
- تتم عملية الإقفال الحصري باستخدام التعليمات lock-X.

2. القفل المشترك (Shared Lock - S Lock):

- يسمح لعدة عمليات بالقراءة من نفس البيانات في الوقت نفسه (إقفال جزئي/مشترك).
- لكنه يمنع التعديل أثناء وجود هذا القفل.
- يستخدم عندما تكون العمليات بحاجة فقط إلى الاطلاع على البيانات دون تغييرها (عملية read).
- تتم عملية الإقفال المشترك باستخدام التعليمات lock-S.

يجري طلب الإقفال من قبل إدارة التحكم في الوصول المتزامن، وتعالج المناقشات فقط بعد تنفيذ طلب الإقفال (منح الإقفال).

⚡ مصفوفة توافق الأقفال (Lock Compatibility Matrix):

تحدد إن كان يمكن منح قفل جديد على عنصر بيانات أم لا، بناءً على الأقفال الموجودة مسبقاً.

القفل المطلوب	القفل المطلوب S	القفل الموجود
S	مسموح	غير مسموح
X	غير مسموح	غير مسموح

	S	X
S	true	false
X	false	false

الاستنتاج:

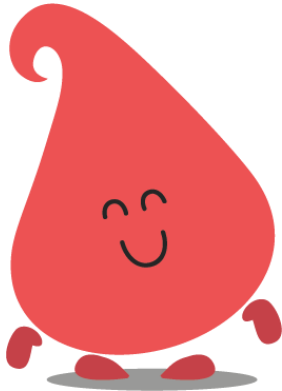
- عدة معاملات يمكنها امتلاك S-lock معاً.
- إذا وُضع X-lock، فلا يُسمح بأي قفل آخر على نفس العنصر.

⚡ ماذا يحدث إذا لم يكن القفل متاحاً؟

إذا طلبت معاملة قفلاً ولم يكن متوافقاً مع الأقفال الموجودة:

- تنتظر المعاملة
- ولا تتابع التنفيذ حتى يتم تحرير الأقفال السابقة (unlock)

مثال: طلب الأقفال



```

T2: lock-S(A);
      read (A);
      unlock(A);
      lock-S(B);
      read (B);
      unlock(B);
      display(A+B)
  
```

ملاحظة مهمة:

هذا التنفيذ غير كافٍ لضمان التسلسلية، لأن:

قد يتم تعديل A أو B بين القراءتين

وبالتالي تكون نتيجة A + B غير صحيحة

لذلك لا يكفي فقط طلب الأقفال، بل نحتاج بروتوكول أقفال واضح

برتوكول الأقفال Locking Protocol:

هو مجموعة قواعد تلتزم بها جميع المعاملات عند:

- طلب الأقفال (lock)
- وتحرير الأقفال (unlock)

البرتوكول يحدد:

- أي مخططات تنفيذ (schedules) مسموحة
- وأيها ممنوعة
- ويضمن تحقيق خاصية التسلسل (Serializability)

مشاكل البروتوكولات المرتكزة على الأقفال:

1. التعارض المتبادل (Deadlock):

مثال:

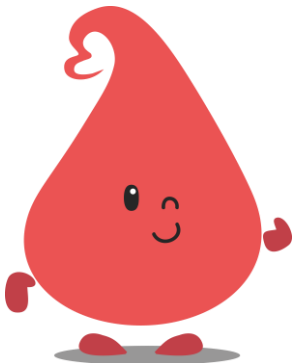
- T3 تطلب lock-X(A) لكنها تنتظر T4
- T4 تطلب lock-S(B) لكنها تنتظر T3

T_3	T_4
lock-X(B) read(B) $B := B - 50$ write(B)	
	lock-S(A) read(A) lock-S(B)
lock-X(A)	

كل معاملة تنتظر الأخرى ولا أحد يتقدم ← Deadlock

الحل:

- اختيار إحدى المعاملتين
- تنفيذ Rollback
- تحرير الأقفال
- السماح للأخرى بالاستمرار



2. الحرمان (Starvation):

حالة لا تستطيع فيها معاملة الحصول على القفل المطلوب أبداً (عدم القدرة على الوصول الى حالة اكتفاء).

مثال:

- معاملة T_i تطلب قفلاً حصرياً (X-lock) على عنصر Q
- في نفس الوقت:
 - معاملات أخرى تطلب أقفالاً مشتركة (S-lock) على Q
 - ويتم منحها هذه الأقفال
- طالما يوجد S-Lock:
 - لا يمكن منح X-Lock.
- فتظل المعاملة T_i منتظرة بلا نهاية

السبب: تصميم غير عادل لآلية منح الأقفال (تصميم إدارة التحكم المتزامن غير جيد)

➤ كيفية تجنب Starvation:

يمكن تصميم نظام التحكم بالتزامن بحيث:

عند طلب معاملة T_i قفلاً على عنصر Q:

1. لا يُمنح القفل إذا:
 - وُجد قفل متناقض مع نوع القفل المطلوب عليه
 2. ولا يُمنح أيضاً إذا:
 - وُجدت معاملة أخرى طلبت القفل قبله وتنتظر.
- أي نراعي أسبقية الطلبات (Fairness)

بروتوكول الإقفال على مرحلتين (2PL) Two-Phase Locking Protocol

هو بروتوكول تحكم بالتزامن يضمن أن جميع مخططات التنفيذ الناتجة عنه متسلسلة تصادمية

(Conflict-Serializable Schedules).

الفكرة بشكل عام هنا أن:

كل معاملة تمر بمرحلتين فقط في حياتها بالنسبة للأقفال:

- تأخذ أقفال فقط.
- ثم تحرر أقفال فقط
- ولا يُسمح بالخلط بين المرحلتين

المرحلة الأولى: مرحلة النمو Growing Phase

في هذه المرحلة:

- المعاملة تحصل على الأقفال S-lock أو X-lock

لا يسمح لها بتحرير أي قفل

أي: عدد الأقفال يزداد أو يبقى ثابتاً لكن لا ينقص أبداً

المرحلة الثانية: مرحلة التقلص Shrinking Phase

في هذه المرحلة:

المعاملة تبدأ بتحرير الأقفال

لا يسمح لها بطلب أي قفل جديد

أي: عدد الأقفال ينقص ولا يمكن أن يزيد مرة أخرى

قاعدة: بمجرد أن تحرر المعاملة أول قفل ← تدخل مرحلة التقلص نهائياً ولا يحق لها طلب أي قفل جديد بعد ذلك.

↙ نقطة القفل Lock Point:

هي: اللحظة التي تحصل فيها المعاملة على آخر قفل تحتاجه

أهمية Lock Point:

ترتيب المعاملات حسب Lock Point يعطينا ترتيباً تسلسلياً مكافئاً

لذلك:

2PL يضمن Conflict Serializability

ملاحظة مهمة:

بروتوكول الإقفال على مرحلتين لا يمنع ال Deadlock، لماذا؟

لأن المعاملات قد:

1. قد تحجز أقفالاً

2. تنتظر أقفالاً أخرى

فينشأ انتظار لا نهائي

مثال:

T3	T4
Lock-X(B) Read(B) B:=B-50 Write(B)	
	lock-S(A) Read(A) Lock-S(B)
Lock-X(A)	

هنا T3 و T4 تحققان بروتوكول 2PL ولكن:

كل معاملة تحجز قفلاً وتنتظر قفلاً لدى الأخرى ← Deadlock

❖ مشكلة أخرى Cascading Rollback:

حتى مع 2PL:

- قد تقرأ معاملة قيمة
- كُتبت من معاملة لم تعمل Commit بعد
- فإذا فشلت المعاملة الأولى:
يجب Rollback للثانية وربما لسلسلة معاملات بعدها

هذه الحالة هي Cascading Rollback

الحل: Strict Two-Phase Locking البروتوكول التام للأقفال على مرحلتين

Strict 2PL

هو نسخة معدلة من 2PL، ويضيف شرطاً مهماً:

- لا يتم تحرير أي Exclusive Lock (X-Lock) (الشرط فقط على القفل الحصري) إلا بعد Commit أو Rollback

النتيجة:

- يمنع Cascading Rollback
- يبقى Conflict Serializable
- لكن Deadlock ما زال ممكناً

مستوى أدق من البروتوكول التام هو: البروتوكول الصارم للأقفال على مرحلتين Rigorous Two-Phase Locking

Rigorous 2PL

في هذا البروتوكول:

- لا يتم تحرير أي قفل (S أو X) (الشرط هنا على نوعي الأقفال) إلا بعد Commit أو Abort

الميزة:

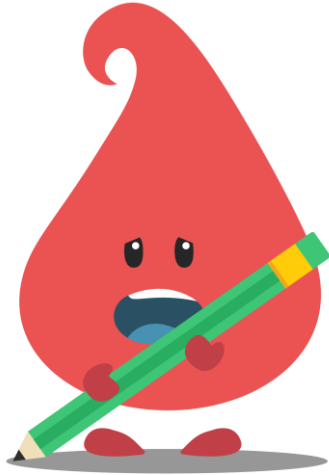
- ترتيب تنفيذ المعاملات = ترتيب ال Commit

مثال:

مخطط تنفيذ يعتمد بروتوكول الإقفال على مرحلتين:

T_5	T_6	T_7
lock-X(A) read(A) lock-S(B) read(B) write(A) unlock(A)	lock-X(A) read(A) write(A) unlock(A)	lock-S(A) read(A)

شرح مخطط التنفيذ:

**T5:**

- أخذ قفل حصري X على A يسمح بالقراءة والكتابة.
- قرأ A.
- أخذ قفل مشترك S على B وقرأه.
- عدّل قيمة (write(A)) A.
- حرّر القفل على A قبل أن يعمل commit

بعد أن حرّر T5 القفل على A:

- T6 يحصل على قفل حصري X على A.
- يقرأ قيمة A (وهي ناتجة عن كتابة T5)
- يعدّل A ثم يحرّر القفل

نلاحظ أنه إذا T5 لاحقاً فشلت، ف T6 تكون قد قرأت بيانات غير مؤكدة (dirty data).

T7:

- يأخذ قفل مشترك S على A.
- يقرأ A بعد أن عدّلها T6 (والتي اعتمدت على T5)

نلاحظ أنه في حال حدوث عطل في T5 بعد A read فإنه يجب عمل rollback ل T5

لكن:

- T6 قرأ A بعد تعديل T5
- T7 قرأ A بعد تعديل T6

ومنه:

- T5 Rollback == T6 rollback
- T6 Rollback == T7 rollback

سيحدث CASCADING ROLLBACK عودة بشكل شلال.

يمكن تجنب هذه المشكلة عن طريق استخدام البروتوكول التام للإقفال على مرحلتين (strict 2PL) أي عن طريق عدم تحرير الأقفال من النوع X إلا بعد حدوث commit أو rollback.

إذا لو كان T5 يستخدم Strict 2PL:

- لن يحرّر lock-X(A) إلا بعد commit
- T6 و T7 لن يتمكنوا من قراءة A قبل التأكد
- لن توجد Dirty Reads
- لن توجد Cascading Rollback

الحياة لا تنتظر استعدادك الكامل، ابدأ بما تملك الآن

التحصيل الآلي للأقفال Automatic Locking

يعني أن: المعاملة لا تطلب الأقفال بشكل صريح
بل نظام إدارة قواعد البيانات يقوم تلقائيًا بالحصول على القفل المناسب عند تنفيذ:

- read(D)
- write(D)

المبرمج يكتب read و write فقط، والنظام يتكفل بالباقي.

التحصيل الآلي عند القراءة (read):

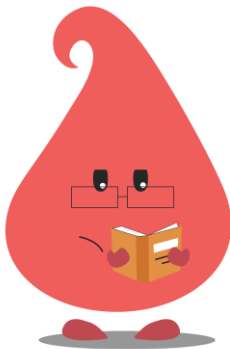
```

if  $T_i$  has a lock on  $D$  then
    read( $D$ )
else
    begin
        if necessary
            wait until no other transaction has a lock-X on  $D$ 
            grant  $T_i$  a lock-S on  $D$ ;
            read( $D$ )
    end
  
```

الشرح:

عندما تطلب المعاملة T_i قراءة العنصر D :

- إذا كان لديها قفل مسبق على D تتم القراءة مباشرة
- إذا لم يكن لديها قفل:
 - تنتظر إذا وُجد قفل حصري X من معاملة أخرى
 - بعد زواله:
 - يُمنح لها قفل مشترك S
 - ثم تنفذ عملية القراءة



السبب:

- القراءة تحتاج lock-S
- يمنع التداخل مع الكتابة.

لا تبحث عن الدافع، التزم وسيأتي الدافع لاحقاً

التحصيل الآلي عند الكتابة (write):

```

if  $T_i$  has a lock-X on  $D$  then
    write( $D$ )
else
    begin
        if necessary
            wait until no other trans. has any lock on  $D$ ,
        if  $T_i$  has a lock-S on  $D$  then
            upgrade lock on  $D$  to lock-X
        else
            grant  $T_i$  a lock-X on  $D$ 
            write( $D$ )
    end;

```

الشرح:

عندما تطلب المعاملة T_i الكتابة على D :

- إذا كان لديها قفل حصري X تكتب مباشرة
- غير ذلك:
 - تنتظر حتى لا يكون هناك أي قفل (S أو X)
 - إذا كانت تملك $lock-S$:
 - يتم ترقية القفل (upgrade) من S إلى X
 - وإلا:
 - يُمنح لها $lock-X$
 - ثم تُنفذ الكتابة

السبب: الكتابة تحتاج تحكماً كاملاً $lock-X ==$

متى تُحرر الأقفال؟

بعد abort أو commit

تنفيذ الإقفال (Lock Manager)**من المسؤول عن الأقفال؟**

يوجد مكوّن في النظام اسمه: Lock Manager

وظيفته (معالجة منفصلة لطلبات الإقفال والتحرير المرسله من قبل المناقلات) أي:

- استقبال طلبات الأقفال من المعاملات
- منح القفل أو جعل المعاملة تنتظر
- تحرير الأقفال عند commit / abort



كيف تتواصل المعاملة مع Lock Manager ؟

- المعاملة ترسل طلب قفل.
- Lock Manger :
 - إمّا يمنح القفل
 - أو يضع المعاملة في حالة انتظار
- في حال التعارض أو deadlock :
 - قد يُجبر معاملة على rollback

Lock Manager يستخدم بنية بيانات اسمها: Lock Table

ماذا يسجل في lock table ؟

لكل عنصر بيانات:

- نوع القفل: (S أو X)
- المعاملات التي تملك القفل (Granted)
- المعاملات المنتظرة (Waiting)

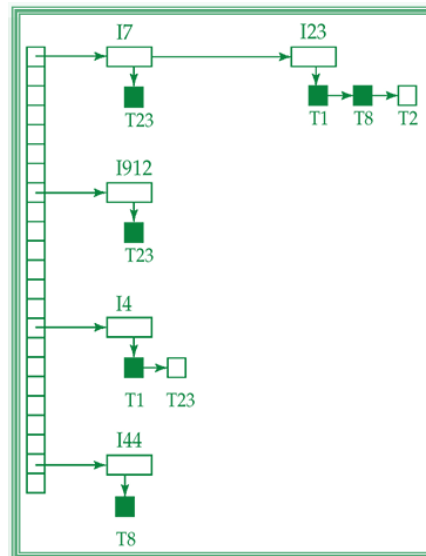
عادة يُنظّم جدول الأقفال (Lock Table) كجدول مُفهرس يعتمد على حقول معطيات العنصر المطلوب إقفاله (أي أن مفتاح الفهرسة هو العنصر المطلوب إقفاله).

مثال:

إذا طلبت معاملة قفل العنصر A، Lock Manager :

- لا يفحص كل جدول الأقفال
- بل يستخدم الفهرس للوصول مباشرة إلى:
 - مدخل العنصر A في Lock Table
 - ويرى:
 - من يملك القفل؟
 - نوع القفل؟
 - هل يوجد معاملات منتظرة؟

كيف يُمثّل جدول الأقفال؟





المستطيلات السوداء = الأقفال الممنوحة (Granted)

المستطيلات البيضاء = طلبات الأقفال المنتظرة (Waiting)

⚡ كيف يُعالج طلب قفل جديد؟

1. يُضاف الطلب إلى نهاية قائمة الطلبات
2. إذا كان متوافقاً مع الأقفال الحالية (الموجودة سابقاً):

- يُمنح فوراً

3. عند تحرير قفل:

- يُعاد فحص الطلبات المنتظرة
- ويُمنح ما أصبح ممكناً منها

⚡ ماذا يحدث عند abort؟

- تحذف كل الأقفال الخاصة بالمعاملة
- تحذف الطلبات المنتظرة لها
- تمنح الأقفال لمعاملات أخرى إن أمكن

يمكن لل Lock Manager الاحتفاظ بقائمة جميع الأقفال التي تملكها كل معاملة.

البروتوكولات المرتكزة على البيان Graph-Based Protocols

هي بديل لبروتوكولات الإقفال ثنائي المراحل (2PL) وتعتمد على بنية بيانية (Graph) بدل الاعتماد على مرحلتي الإقفال. تعتمد هذه البروتوكولات على ترتيب جزئي للمعطيات لضمان تنفيذ آمن للمعاملات أي: بفرض وجود ترتيب جزئي على عناصر المعطيات:

$$D = \{d_1, d_2, \dots, d_n\}$$

هذا الترتيب ناتج عن:

- التنظيم الفيزيائي للمعطيات
 - أو التنظيم المنطقي (مثل شجرة، فهرس، ملفات..)
- إذا كانت هناك علاقة ترتيب:

$$d_i \rightarrow d_j$$

فإنه: أي مناقلة تريد استخدام d_i و d_j معاً يجب أن تصل إلى d_i قبل d_j

- تمثل عناصر المعطيات D على شكل:

- بيان موجّه دون حلقات (Directed Acyclic Graph (DAG)

هذا البيان يسمى: Database Graph

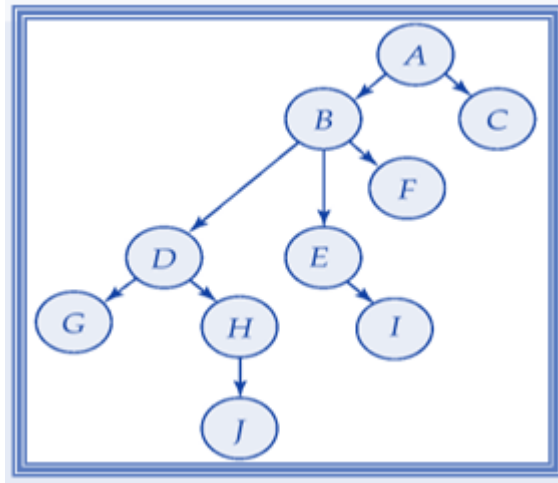
من لم يتعب في البناء، سيتعب طويلاً في الترميم

لماذا بدون حلقات؟

لأن:

- وجود حلقة يعني إمكانية الوصول المتبادل
- وهذا قد يؤدي إلى:
 - تعارض
 - أو عدم إمكانية تحقيق تسلسل تصادمي مضمون

حالة خاصة مبسطة من بروتوكولات البيان Tree Protocol:



هو:

أبسط وأشهر مثال على Graph-Based Protocol

حيث:

- يكون Database Graph على شكل شجرة.

قواعد Tree Protocol:

نوع الأقفال

- يسمح فقط بالأقفال الكلية Exclusive Locks (X)

القاعدة الأولى:

يمكن للمعاملة T_i أن تطلب قفل أي عنصر بيانات كأول قفل لها

القاعدة الثانية:

لاحقاً، لا يمكن لـ T_i قفل عنصر Q إلا إذا كان والده (Parent) في الشجرة مقفولاً مسبقاً من قبل T_i

القاعدة الثالثة:

يمكن تحرير الأقفال في أي وقت، لكن:

لا يمكن إعادة قفل عنصر تم تحرير قفله سابقاً من نفس المعاملة

جميع المخططات التي يحققها Tree Protocol: هي مخططات متسلسلة تصادمية

البروتوكولات المرتكزة على الأختام (Timestamp-Based Protocols)

في هذا البروتوكول:

كل معاملة T_i تُعطى عند دخولها النظام ختم زمني (Timestamp): $TS(T_i)$ بحيث:

■ إذا دخلت T_i قبل T_j فإن: $TS(T_i) < TS(T_j)$

والهدف من ذلك: تنفيذ المعاملات كما لو كانت متسلسلة حسب ترتيب الختم الزمني

يعني: النظام يفرض الترتيب الزمني بدل الأقفال

لتحقيق هذا الهدف يقوم البروتوكول بالتالي:

لكل عنصر بيانات Q نخزن قيمتين للختم:

1. $R\text{-timestamp}(Q)$: أكبر ختم زمني لمناقلة قرأت Q بنجاح (الختم الزمني لآخر مناقلة قرأت Q)
2. $W\text{-timestamp}(Q)$: أكبر ختم زمني لمناقلة كتبت Q بنجاح (الختم الزمني لآخر مناقلة كتبت Q)

قواعد البروتوكول:

تنفيذ $read(Q)$ من قبل المناقلة T_i :

القاعدة 1: If $TS(T_i) < W\text{-timestamp}(Q)$

- مرفوض.
- لأن Q كُتب من معاملة أحدث من T_i
- القراءة تكسر الترتيب الزمني
- النتيجة: $rollback(T_i)$

القاعدة 2: If $TS(T_i) \geq W\text{-timestamp}(Q)$

- مسموح
- تُنفذ القراءة
- نحدّث قيمة أحدث قراءة: $R\text{-timestamp}(Q) = \max(R\text{-timestamp}(Q), TS(T_i))$

تنفيذ $write(Q)$ من قبل المناقلة T_i :

القاعدة 1: If $TS(T_i) < R\text{-timestamp}(Q)$

- مرفوض
- عنصر Q قُرئ من معاملة أحدث
- الكتابة ستخرب الترتيب الزمني
- النتيجة: $rollback(T_i)$

If $TS(T_i) < W\text{-timestamp}(Q)$

القاعدة 2:

- مرفوض
- توجد كتابة أحدث على Q
- هذه الكتابة أصبحت غير مهمة (مهملة)
- النتيجة: $rollback(T_i)$

Else

القاعدة 3:

- مسموح
- تنفذ الكتابة
- نحدث قيمة آخر كتابة: $W\text{-timestamp}(Q) = TS(T_i)$

مثال حول استخدام البروتوكول:

ليكن لدينا مخطط جزئي من مخطط تنفيذ مجموعة من المناقلات مع أختام بقيم 1,2,3,4,5 أي:

■ $TS(T_4) = 4$

■ $TS(T_5) = 5$

■ $TS(T_1) = 1$

■ $TS(T_2) = 2$

■ $TS(T_3) = 3$

T_1	T_2	T_3	T_4	T_5
				read (X)
read (Y)	read (Y)			
		write (Y) write (Z)		
	read (Z) abort			read (Z)
read (X)			read (W)	
		write (W) abort		
				write (Y) write (Z)

يبدأ المثال بافتراض أن جميع قيم:

■ $R\text{-TS}(Q) = 0$

■ $W\text{-TS}(Q) = 0$

لكل عنصر بيانات Q (أي لا قراءة ولا كتابة تمت بعد)

قد تتأخر النتائج، لكن التعب الصادق لا يضيع أبداً

تنفيذ العمليات حسب الترتيب في المخطط:

1. T1 : read(Y)

$$W-TS(Y) = 0$$

$$TS(T1) = 1 \geq 0$$

القراءة مسموحة

$$R-TS(Y) = \max(0, 1) = 1 \text{ نحدّث:}$$

2. T2 : read(Y)

$$W-TS(Y) = 0$$

$$TS(T2) = 2 \geq 0$$

القراءة المسموحة

$$R-TS(Y) = \max(1, 2) = 2 \text{ نحدّث:}$$

3. T3 : write(Y)

نطبّق شروط الكتابة:

$$TS(T3) = 3 \geq R-TS(Y) = 2 \text{ محقق}$$

$$TS(T3) = 3 \geq W-TS(Y) = 0 \text{ محقق}$$

الكتابة مسموحة

$$W-TS(Y) = 3 \text{ نحدّث:}$$

4. T3 : write(Z)

$$W-TS(Z) = 0, R-TS(Z) = 0$$

$$TS(T3) = 3 \geq 0$$

الكتابة مسموحة

$$W-TS(Z) = 3 \text{ نحدّث:}$$

5. T5 : read(X)

$$W-TS(X) = 0$$

$$TS(T5) = 5 \geq 0$$

القراءة مسموحة

$$R-TS(X) = 5 \text{ نحدّث:}$$

T2 : write(X) .6

نختبر شروط الكتابة:

$$TS(T2) = 2$$

$$R-TS(X) = 5$$

$$2 < 5 = \text{خرق للقاعدة}$$

Abort T2

السبب: T2 يحاول كتابة X بعد أن قرأته معاملة أحدث (T5).

T5 : read(Z) .7

$$W-TS(Z) = 3$$

$$TS(T5) = 5 \geq 3$$

القراءة مسموحة

$$\text{نحدث: } R-TS(Z) = 5$$

T4 : read(W) .8

$$W-TS(W) = 0$$

$$TS(T4) = 4 \geq 0$$

القراءة مسموحة

$$\text{نحدث: } R-TS(W) = 4$$

T3 : write(W) .9

نختبر شروط الكتابة:

$$TS(T3) = 3$$

$$R-TS(W) = 4$$

$$3 < 4$$

Abort T3

السبب: T3 يحاول كتابة W بعد أن قرأته معاملة أحدث (T4).

T5 : write(Y) .10

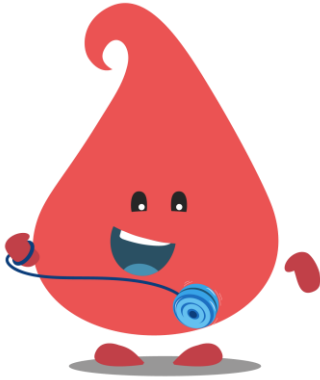
$$R-TS(Y) = 2$$

$$W-TS(Y) = 3$$

$$TS(T5) = 5 \geq 2 \text{ و } 3 \leq$$

الكتابة مسموحة

$$\text{نحدث: } W-TS(Y) = 5$$



■ نلاحظ أن T3 كتب:

• write(Y)

• write(Z)

■ ثم لاحقاً T3 ثم عمل (Abort) له

لكن قبل ال abort:

■ T5 قرأ Z

■ ثم أكمل وكتب write(Y) و write(Z)

حتى نحافظ على صحة القاعدة:

لا يجوز لـ T5 أن يستمر وكأن شيئاً لم يحدث، لأنه استخدم بيانات مصدرها معاملة فاشلة
يجب إرجاع T5 (Rollback) أيضاً
ومنه يجب العودة بشكل شلال للمحافظة على صحة القاعدة.

مثال آخر حول استخدام البروتوكول:

T_{25}	T_{26}
read(B)	read(B) $B := B - 50$ write(B)
read(A)	read(A)
display(A + B)	$A := A + 50$ write(A) display(A + B)

لدينا معاملتان:

■ T25 حيث: TS(T25) = 25

■ T26 حيث: TS(T26) = 26

وقيم البداية:

■ $R-TS(A) = W-TS(A) = 0$

■ $R-TS(B) = W-TS(B) = 0$

ليست القوة في أن لا تسقط، بل في أن تنهض دون أن تكره الطريق

➤ تطبيق القواعد:

1. T25 : read(B)

قاعدة القراءة:

يُسمح ل $\text{read}(Q)$ إذا كان $\text{TS}(Ti) \geq \text{W-TS}(Q)$

$$\text{TS}(T25) = 25$$

$$\text{W-TS}(B) = 0$$

مسموح

$$\text{R-TS}(B) = \max(0, 25) = 25 \quad \text{نحدّث:}$$

2. T26 : read(B)

$$\text{TS}(T26) = 26$$

$$\text{W-TS}(B) = 0$$

مسموح

$$\text{R-TS}(B) = \max(25, 26) = 26 \quad \text{نحدّث:}$$

3. T26 : write(B)

قاعدة الكتابة:

يُسمح ل $\text{write}(Q)$ إذا:

$$\text{TS}(Ti) \geq \text{R-TS}(Q)$$

$$\text{TS}(Ti) \geq \text{W-TS}(Q)$$

نختبر:

$$\text{TS}(T26) = 26$$

$$\text{R-TS}(B) = 26$$

$$\text{W-TS}(B) = 0$$

الشرطان محققان

$$\text{W-TS}(B) = 26 \quad \text{نحدّث:}$$

4. T25 : read(A)

$$\text{TS}(T25) = 25$$

$$\text{W-TS}(A) = 0$$

مسموح

$$\text{R-TS}(A) = 25 \quad \text{نحدّث:}$$



T26 : read(A) .5

$$TS(T26) = 26$$

$$W-TS(A) = 0$$

مسموح

$$R-TS(A) = 26 \text{ : نحدّث}$$

T26 : write(A) .6

نختبر شروط الكتابة:

$$TS(T26) = 26$$

$$R-TS(A) = 26$$

$$W-TS(A) = 0$$

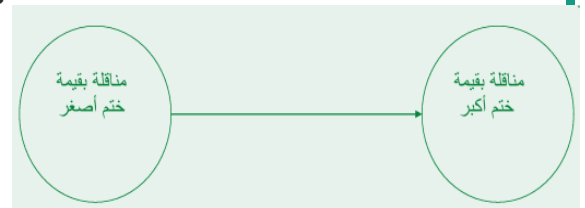
مسموح

$$W-TS(A) = 26 \text{ : نحدّث}$$

نستنتج: هذا المخطط صالح تحت بروتوكول Timestamp based لأن جميع عمليات القراءة والكتابة تحقق شروط المقارنة بين الأختام الزمنية TS مع R-TS و W-TS دون حدوث أي تعارض يؤدي إلى Abort.

Correctness of Timestamp Ordering Protocol صحة بروتوكول ترتيب الأختام

- يؤمن / يضمن بروتوكول ترتيب الأختام التسلسلية (Serializability) حيث أن جميع الأقواس الموجودة في بيان السوابق (precedence graph) هي من الشكل:



حيث أنه لن يكون هناك حلقات في بيان السوابق ← المخطط متسلسل تصادياً (Conflict Serializable).

- يضمن هذا البروتوكول التحرر من Deadlock (مشكلة العرقلة المتبادلة) حيث أن المناقلات لا تنتظر ابداً لأنه في حال حدوث تعارض يتم عمل abort للمناقلة بالتالي لا تدخل في حالة انتظار.
- لا يضمن هذا البروتوكول أن يكون المخطط Cascade-Free (أي يمكن أن يحوي المخطط على مشكلة العودة بشكل شلال).
- لا يضمن هذا البروتوكول Recoverability , أي يمكن أن يكون المخطط الناتج غير قابل للاستعادة.

الاستعادة والتحرر من مشكلة الشلال

Recoverability and Cascade Freedom

المشاكل الموجودة في بروتوكول ترتيب الأختام:

- إذا افترضنا أن المناقلة T_j قامت بقراءة معطيات كتبتها المناقلة T_i قبل أن تُنهي T_i عملها (commit)، ثم قامت T_j بتنفيذ commit، وبعد ذلك حدث abort للمناقلة T_i ، فإن المخطط الناتج يكون غير قابل للاستعادة (Non-Recoverable Schedule)
- في حال حدوث abort للمناقلة ما، يجب إنهاء (abort / rollback) جميع المناقلات التي قرأت عناصر مكتوبة من قبلها، مما يؤدي إلى العودة بشكل شلال (Cascading Rollback)

أحد الحلول المقترحة:

- تنظّم المناقلة بحيث تُنفذ جميع عمليات الكتابة في نهاية تنفيذها.
- تُنفذ جميع كتابات المناقلة كوحدة ذرية واحدة (Atomic Action)، أي:
 - لا يُسمح لأي مناقلة أخرى بالتنفيذ أثناء مرحلة الكتابة في هذه المناقلة.
- في حال حدوث abort، تُعاد المناقلة للتنفيذ بختم زمني جديد

التعامل مع العرقلة المتبادلة Deadlock Handling

لدينا المناقلتان:

- T_1 : write(X) , write(Y)
- T_2 : write(Y) , write(X)

مثال على مخطط تنفيذ يحوي مشكلة العرقلة المتبادلة:

T_1	T_2
lock-X on X write (X)	lock-X on Y write (Y)
wait for lock-X on Y	wait for lock-X on X

T1 تنتظر T2 و T2 تنتظر T1 ← deadlock

لا تجعل خوفك من الفشل يحرمك شرف المحاولة

بروتوكولات منع العرقلة المتبادلة

Deadlock Prevention Protocols

هدفها: منع النظام من الدخول في Deadlock من الأساس

لنذكر بعض استراتيجيات المنع:

1. استراتيجية القفل المسبق (Preclaiming Locks):

- كل مناقلة:
- تطلب جميع الأقفال التي تحتاجها قبل بدء التنفيذ (إقفال جميع العناصر التي ستتعامل معها)
- إذا لم تتوفر جميعها:
- لا تبدأ المناقلة

2. الترتيب الجزئي لعناصر المعطيات (البروتوكول المرتكز على البيان):

- يفترض وجود ترتيب ثابت على عناصر المعطيات
- كل مناقلة:
- تطلب الأقفال وفق هذا الترتيب فقط

3. استراتيجيات تعتمد على الأختام الزمنية:

A. wait-die

- إذا طلبت مناقلة قديمة مورداً من مناقلة أحدث:
- تنتظر
- إذا طلبت مناقلة أحدث مورداً من مناقلة أقدم:
- تقوم بعملية (rollback) بدلاً من الانتظار وهذا ما يسمى بـ die
- الأحدث هي التي تُضحي بها.
- قد يحدث موت المناقلة die عدة مرات قبل أن تحصل على المعطيات التي تحتاجها

سميت بهذا الاسم لأن القرار دائماً أحد خيارين:

wait: إذا كانت المعاملة أقدم

die: إذا كانت المعاملة أحدث

ولا يوجد خيار ثالث

wound-wait .B

- إذا طلبت مناقلة قديمة مورداً من مناقلة أحدث:
 - تجبر الأحداث على rollback
 - إذا طلبت مناقلة أحدث مورداً من مناقلة أقدم:
 - تنتظر
- الأقدم هي التي تفرض سيطرتها

هذه الاستراتيجية تسبب عدداً أقل من عمليات التراجع (rollback) في wait-die

في الاستراتيجيتين السابقتين:

- المناقلة التي تتراجع:
- يُعاد تنفيذها لاحقاً
- بنفس الختم الزمني الأصلي

هذا يساعد على منع ظاهرة starvation الحرمان (لأن المعاملة الأقدم دائماً لها أولوية)

كشف العرقلة المتبادلة

Deadlock Detection

الفكرة هنا: بدل المنع، نسمح بحدوث Deadlock ثم نكشفه

نستخدم بيان الانتظار-من-أجل (Wait-For Graph) للكشف عن العرقلة المتبادلة وهذا البيان يتألف من:

- العقد (V): المناقلات
- الأقواس (E): $T_i \rightarrow T_j$ تعني أن T_i تنتظر T_j لتحرير مورد

يُضاف القوس عند الانتظار

يُحذف عند الحصول على القفل

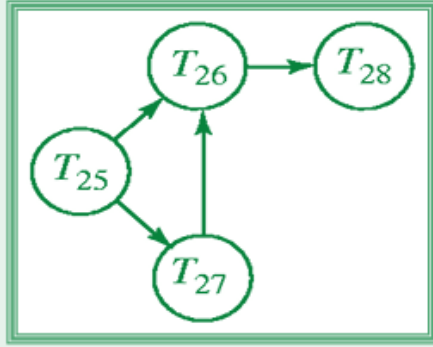
⚡ شرط وجود Deadlock:

وجود حلقة (Cycle) في Wait-For Graph

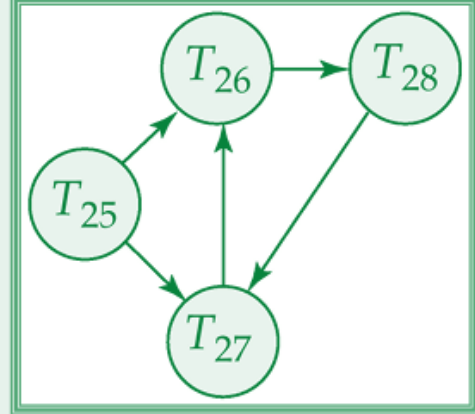
- بدون حلقة ← لا Deadlock
- مع حلقة ← Deadlock مؤكد

⚡ كيف يتم الكشف؟

- تنفيذ خوارزمية كشف العرقلة المتبادلة بشكل دوري
- البحث عن Cycles في Wait-For Graph



بيان "wait-for" بدون حلقة



بيان "wait-for" مع حلقة

إزالة العرقلة المتبادلة

Deadlock Recovery

عند اكتشاف Deadlock:

لإزالة العرقلة:

- نختار مناقلة (أو أكثر) كضحية.
- نقوم بعمل Rollback لها
- نفتح الأقفال
- تكمل باقي المناقلات

كيف نختار المناقلة الضحية؟

نستخدم عوامل حساب الكلفة (Cost Factors) مثل:

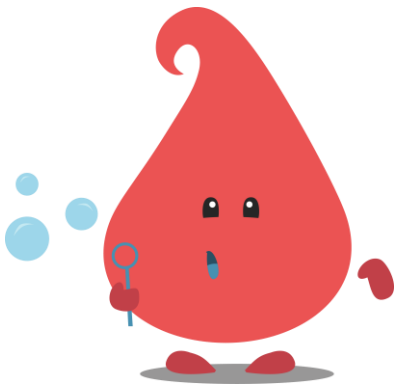
- عدد الأقفال التي تملكها
- مدة تنفيذها
- عدد مرات rollback السابقة
- كمية العمل المنجز

مع ملاحظة أنه إذا اخترنا دائماً نفس المناقلة:

سيؤدي ذلك إلى مشكلة الحرمان starvation

لتجنب الحرمان الحل:

- إدخال عدد مرات rollback كعامل من عوامل حساب الكلفة



عمليات الإضافة والحذف (Insert & Delete Operations)

عند استخدام بروتوكول الإقفال ثنائي المراحل (Two-Phase Locking - 2PL) فإن التعامل مع عمليات الإضافة والحذف يتم وفق القواعد التالية:

- لا يمكن تنفيذ عملية الحذف إلا إذا حصلت المناقلة على قفل حصري/كلي (X-lock) على الحدودية المراد حذفها.
 - كذلك، فإن المناقلة التي تقوم بإضافة سجل جديد إلى قاعدة البيانات يجب أن تحصل على قفل حصري / كلي (X-lock) على الحدودية التي ينتمي إليها السجل الجديد.
- على الرغم من ذلك، فإن عمليات الإضافة والحذف قد تؤدي إلى ظهور مشكلة تعرف باسم:

ظاهرة الأشباح

(Phantom Phenomenon)

تظهر مشكلة الأشباح عندما تقوم مناقلة بقراءة مجموعة من السجلات التي تحقق شرطاً حدياً معيناً، ثم تقوم مناقلة أخرى أثناء تنفيذ الأولى بإضافة أو حذف سجلات جديدة تحقق نفس الشرط، دون أن تكون المناقلة الأولى على علم بهذه التغييرات.

ونتيجة لذلك:

- تتغير مجموعة السجلات التي يفترض أن تقرأها المناقلة الأولى.
- تكون نتيجة القراءة غير ثابتة وقد تختلف عند إعادة تنفيذ نفس الاستعلام.
- لا تلاحظ مناقلة القراءة السجلات الجديدة المضافة أو المحذوفة، لأنها لم تكن موجودة عند بداية تنفيذها.

مثال توضيحي:

لدينا المناقلتان التاليتان:

T1

```
SELECT SUM(balance)
FROM account
WHERE branch-name = "perryridge";
```

T2

```
INSERT INTO account
VALUES ("perryridge", 9786, "Smith", 1200);
```

في حال تنفيذ المناقلتين بشكل متداخل، قد تقوم T1 بحساب مجموع الأرصدة دون أن ترى السجل الجديد الذي أضافته T2، رغم أنه يحقق نفس شرط الفرع، مما يؤدي إلى نتيجة غير صحيحة وغير متسلسلة. وهذا ما يُعرف بـ Phantom Phenomenon، حيث تظهر (أو تختفي) سجلات جديدة أثناء تنفيذ المناقلة.

إضاءة أريسيزية توضيحية

الحدوديات (Predicates):

هي شروط منطقية تحدد مجموعة محتملة من الصفوف، سواء كانت موجودة الآن أو قد تُضاف لاحقاً.

مثال:

branch = "perryridge"

هذه الحدودية لا تمثل صفاً معيناً، بل تمثل: كل الصفوف التي تحقق هذا الشرط الآن أو في المستقبل

سبب ظاهرة الأشباح:

1. الإقفال التقليدي يقتصر على السجلات (الصفوف) الموجودة فعلياً فقط:

- هذا النوع من الإقفال لا يمنع مناقلات أخرى من:
- إضافة سجلات جديدة لاحقاً.
- حذف سجلات تحقق نفس شرط الاستعلام.

بعبارة أخرى:

- الأقفال التقليدية تقفل الصفوف نفسها.
- لكنها لا تقفل نطاق الشرط الحدي.
- لذلك قد تظهر سجلات جديدة "شبحية" لم تكن موجودة عند القراءة الأولى

ونتيجة لذلك:

- قد ترى مناقلة القراءة أو المسح مجموعات مختلفة من السجلات عند إعادة تنفيذ نفس الاستعلام

2. كما أن الاكتفاء باستخدام الأقفال على الحدوديات دون آلية إضافية للتحكم بالوصول قد يؤدي إلى توليد:

مخططات تنفيذ غير متسلسلة تصادمياً (Not Conflict Serializable Schedules)

حل مشكلة الأشباح:

1. استخدام الأقفال على الحدوديات (Predicate Locks)

- تقوم المناقلة القارئة بحجز قفل مشترك (S-lock) على الشرط الحدي أثناء القراءة.
- تقوم مناقلة الإضافة أو الحذف بطلب قفل حصري (X-lock) على نفس الشرط.
- يمنع ذلك تنفيذ عمليات الإضافة أو الحذف حتى انتهاء القراءة.

العيب: هذا الحل يؤدي إلى انخفاض كبير في درجة التزامن (Concurrency)

من عرف غايته، هانت عليه مشقة الطريق

2. بروتوكول إقفال الفهارس (Index Locking Protocols):

يُعد هذا البروتوكول حلًا عمليًا لمشكلة الأشباح مع الحفاظ على درجة أعلى من التزامن.

- يجب أن تكون كل علاقة مرتبطة على الأقل بفهرس واحد.
- أي عملية وصول إلى العلاقة يجب أن تتم من خلال الدليل/الفهرس.
- يتم تطبيق الأقفال على كتل الفهرس (Index Blocks) بدلاً من تطبيقها على الشروط الحديثة بشكل كامل.

⚡ كيف تُقفل العلاقة باستخدام الفهرس؟

1. مناقلة المسح (المناقلة التي تقوم بالقراءة) (Transaction that reads the relation):

تحصل على أقفال من نوع: S-mode (Shared Locks) على كتل الفهرس التي تُستخدم للوصول إلى السجلات المطلوبة. هذا يمنع أي تعديل على الفهرس أثناء القراءة، لكنه يسمح لقراء آخرين بالوصول.

2. مناقلة الإضافة أو الحذف (Insert / Delete Transaction):

يجب على هذه المناقلة تعديل الفهارس المرتبطة بالعلاقة.

تحصل على أقفال من نوع: X-mode (Exclusive Locks) على:

- جميع كتل الفهرس التي ستتغير
- أو الكتل التي قد تحتوي على الحدود الجديدة بعد الإضافة أو الحذف

بمعنى:

- عند الإضافة: يجب قفل الكتلة التي سيُضاف فيها السجل
- عند الحذف: يجب قفل الكتلة التي يُحذف منها السجل
- وفي كلتا الحالتين: يجب قفل أي كتلة قد تتأثر بالحدود (Predicates)

The End...

من لم يدفع ثمن حلمه تعبًا، سيدفع ثمن نسيانه ندمًا